AGENTIC SYSTEMS · LECTURE

Agentic AI

From prompts to controlled, goal-directed action

ReAct · LangGraph
Inference Stack · MCP
Operational Readiness

PLAN → ACT → OBSERVE → UPDATE



Prof. Amlan Chakrabarti

Department of Artificial Intelligence
IIT Kharagpur

3 November 2026

Two-hour roadmap

A systems-oriented path from the agent concept to production readiness

01 Foundations + ReAct

- Agent definition and loop
- Reason–Act–Observe traces

02 LangGraph + orchestration

- State, nodes, edges
- Cycles, checkpoints, approvals

03 RAG + inference

- Retrieval and grounding
- Prefill, decode, KV cache

04 MCP + interoperability

- Tools, resources, prompts
- MCP vs A2A

Concepts › Build › Serve › Govern

Learning outcomes

By the end, learners should be able to explain, implement and evaluate agentic workflows

01 Explain the agent loop

- Define goal, state, policy and observation
- Specify termination and safe-stop conditions

02 Build tool-using workflows

- Create typed tools with validated inputs
- Implement conditional graph routing

03 Integrate capabilities

- Use RAG for evidence
- Expose tools through MCP; delegate with A2A

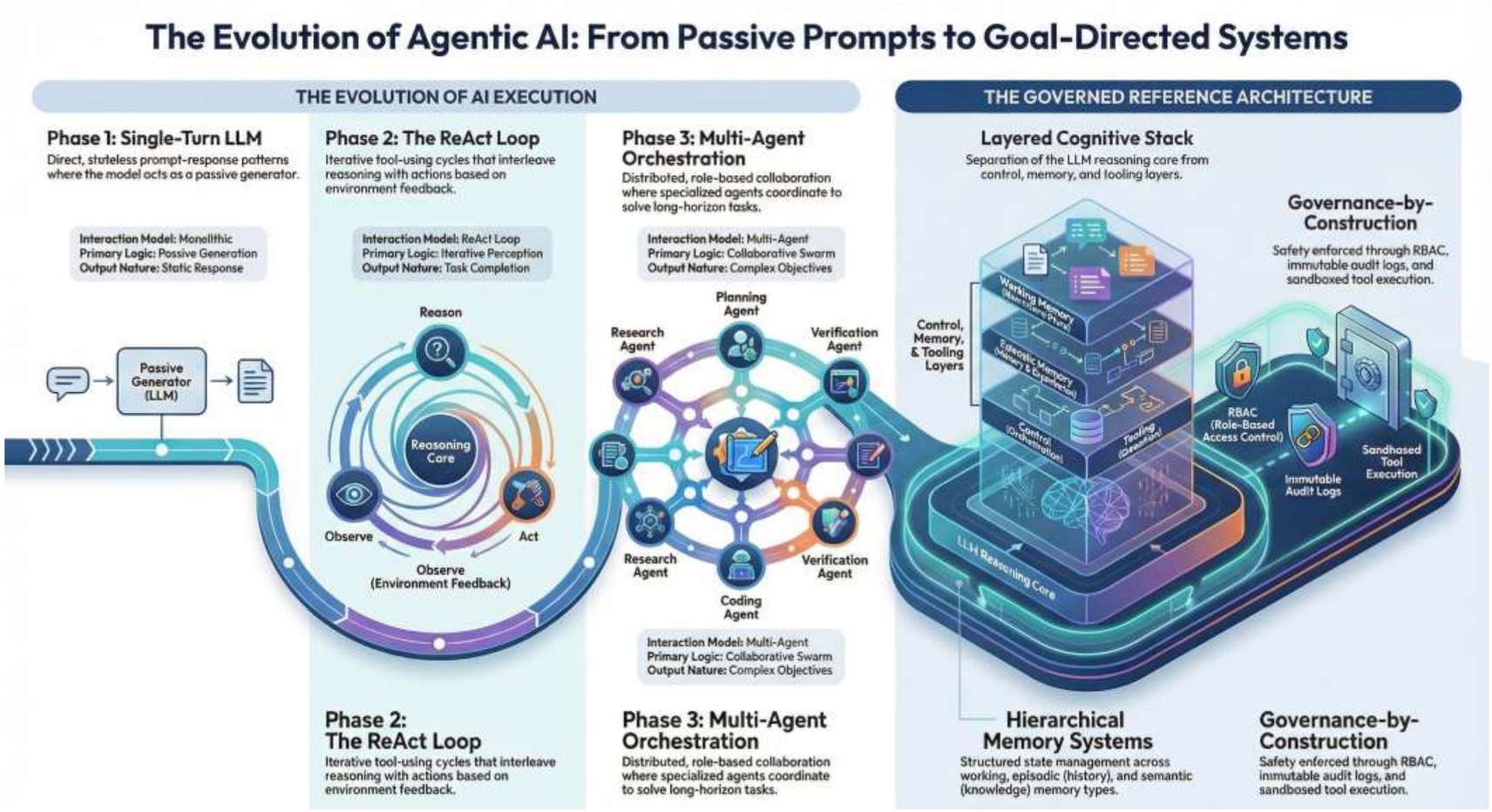
04 Engineer for production

- Reason about latency and memory
- Evaluate traces, approvals and failure recovery

Model › Graph › Tools + RAG › Inference › Governance

From prompt–response to goal-directed systems

The key shift is from producing one answer to controlling a multi-step task trajectory



Mamdouh Alenezi.arXiv:2602.10479v1

Prompt–response and goal-directed agents

The key shift is from producing one answer to controlling a multi-step task trajectory

01 Prompt–response

- Input → model → answer
- Reactive, usually one generation
- No explicit external state transition

02 Goal-directed agent

- Goal → plan → action → observation
- Uses tools and persistent state
- Repeats until verified completion or safe stop

Agent trajectory: $\tau = (s_0, a_0, o_1, \ldots, a_{\square}, o_{\square+1})$

The system is evaluated on the complete trajectory and final outcome—not only the text response.
Mamdouh Alenezi.arXiv:2602.10479v1

LLM, RAG, MCP and Agent: distinct roles

These components are complementary; they solve different architectural problems

01 LLM — reasoner

- Maps context to generated tokens
- Provides language understanding and planning proposals
- Does not inherently execute external actions

02 RAG — knowledge access

- Retrieves relevant external evidence
- Adds domain context and provenance
- Does not itself orchestrate a task

03 MCP — capability interface

- Standardizes discovery and invocation of tools/resources
- Connects hosts to external systems
- Does not grant authorization

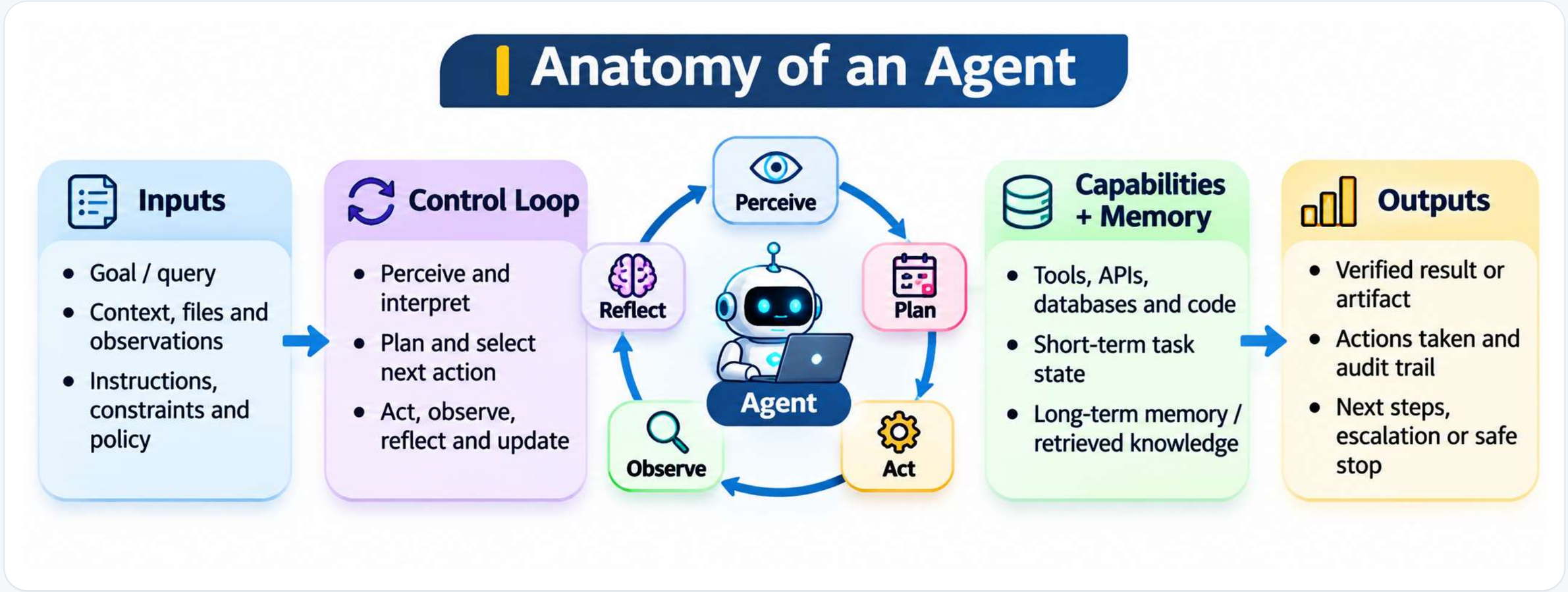
04 Agent — orchestrator

- Maintains goal and state
- Chooses when to retrieve or call tools
- Coordinates actions and verifies completion

Goal › Agent › LLM + RAG + MCP › Verified result

Anatomy of an agent

An agent combines inputs, a control loop, memory and external capabilities



Input › Perceive › Plan › Tool / Act › Observe › Update

WORKING BUSINESS EXAMPLE

Retail order-resolution agent

Use case: Resolve a delayed customer order by coordinating order, inventory and carrier systems

CUSTOMER GOAL

“My order is late. Please tell me where it is and help me get it as soon as possible.”

INPUTS Order ID • customer profile • promised delivery date • service policy • current shipment status

1 PERCEIVE

Identify the order
Detect late-delivery risk
Read customer and policy context

2 PLAN

Check order and payment
Check stock and alternatives
Choose the next best action

3 TOOL / ACT

Query order-management API
Check warehouse inventory
Request carrier tracking update

4 OBSERVE

Carrier reports a hub delay
Nearby store has stock
Revised ETA is available

5 UPDATE

Compare recovery options
Apply policy and cost limits
Seek approval if required

FEEDBACK LOOP If the carrier cannot confirm delivery → re-check inventory / offer pickup or replacement → update the plan.

Customer update
Clear status + revised ETA

Recovery option
Reship • pickup • refund

Audit trail
API calls • decisions • approvals

Safe completion
Policy-compliant action or escalation

Agentic AI in business operations • The agent coordinates systems and verifies outcomes; it does not merely generate a customer-service reply.

The agent loop as an algorithm

Bound the loop with explicit policy checks, typed tools and a maximum step budget

PYTHON / IMPLEMENTATION PATTERN

```
state = initialise(goal)
for step in range(MAX_STEPS):
    context = build_context(state, memory, policy)
    action = model.propose(context)

    if not policy.allows(action):
        action = repair_or_escalate(action)

    if action.is_tool_call():
        args = validate_schema(action.args)
        result = tools.execute(args, timeout=10)
        state = update(state, action, result)
        memory.write(state)
    else:
        return verify_and_finalize(action, state)

return safe_stop(state)
```

Why the guardrails matter

1. MAX_STEPS prevents unbounded loops.
2. Policy gates authorize every action.
3. Typed schemas constrain tool inputs.
4. State checkpoints support recovery.
5. Safe stop returns partial work or escalates.

Tool calling: a typed contract

Treat every tool as a small, versioned API—not as arbitrary model-generated code

PYTHON / IMPLEMENTATION PATTERN

```
from pydantic import BaseModel, Field

class SearchArgs(BaseModel):
    query: str = Field(min_length=3, max_length=240)
    limit: int = Field(default=5, ge=1, le=20)

def search_papers(args: SearchArgs) -> list[dict]:
    return paper_index.search(
        query=args.query,
        limit=args.limit
    )
```

Contract checklist

1. Schema: types, ranges, required fields
2. Identity: who may invoke it?
3. Scope: which records/resources?
4. Execution: timeout, rate limit, idempotency
5. Output: structured result + provenance
6. Failure: typed errors and safe fallback

Never execute arbitrary model-generated shell commands or code in the host process.

ReAct: a worked execution trace

Reasoning and acting are interleaved; observations influence the next decision

01 Reason

- Need current evidence
- Select a search action
- State: evidence is missing

02 Act

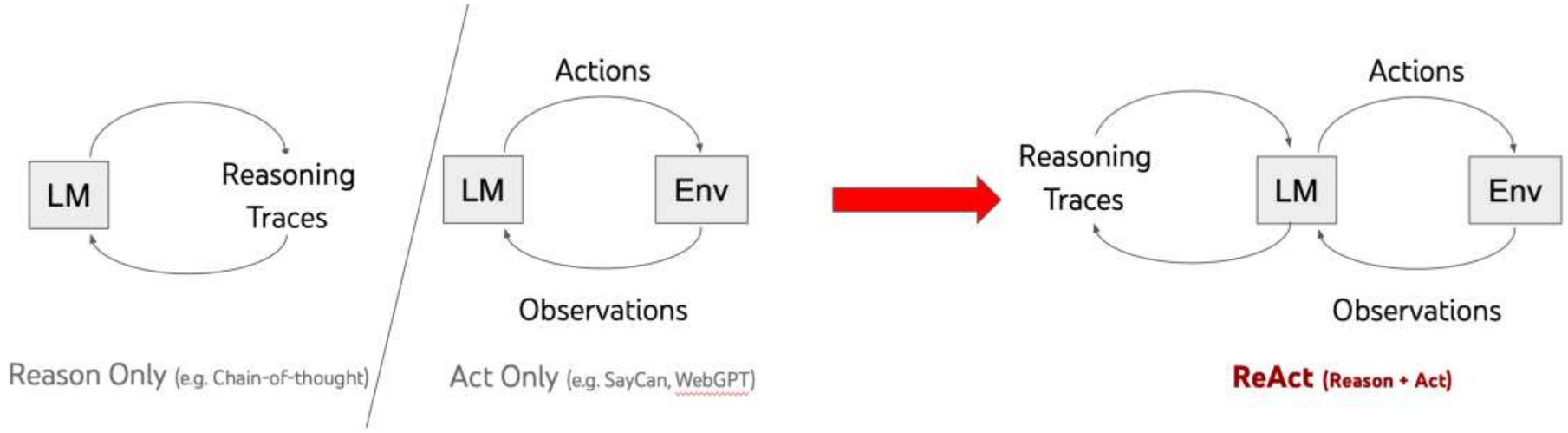
- Call search_papers(query)
- Tool executes outside the LLM
- Arguments are schema-validated

03 Observe

- Three papers returned
- Inspect source quality and dates
- Update the task state

04 Reason again

- Open the strongest source
- Extract method and results
- Verify claims before synthesis



ReAct: interleaved reasoning traces and actions — Yao et al., ICLR 2023 (arXiv:2210.03629)

Reason → Act → Observe → Reason → Act → Observe

Code 1: a minimal tool-calling pattern

The model selects a tool; the application executes it and feeds the result back

PYTHON / IMPLEMENTATION PATTERN

```
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

@tool
def get_lab_status(lab: str) -> str:
    """Read status for an approved lab."""
    return status_api.read(lab)

tools = [get_lab_status]
model = ChatOpenAI(model="<pinned-model>", temperature=0)
agent_model = model.bind_tools(tools)

response = agent_model.invoke([
    ("system", "Use tools only for lab-status questions."),
    ("human", "Is the FPGA lab operational?")
])
# Application dispatcher:
# 1. inspect response.tool_calls
# 2. authorize + validate each call
# 3. execute tool and append ToolMessage
# 4. invoke model again with the observation
```

What happens

1. @tool exposes a schema and description.
2. bind_tools supplies tool definitions to the model.
3. The application—not the model—executes the function.
4. Tool output becomes a new observation.
5. Repeat until answer or stop condition.

ReAct failure modes: detect, recover, stop

A robust agent does not blindly repeat a failed action

01 Empty result

- Broaden query once
- Ask user to clarify if still empty

02 Timeout / transient error

- Retry with exponential backoff
- Use a strict retry budget

03 Invalid arguments

- Validate schema
- Repair once; log repeated failure

04 Conflicting evidence / no progress

- Rank source quality and expose disagreement
- Detect repeated state–action pairs and stop

Attempt › **Validate** › **Bounded retry** › **Escalate / stop**

RAG is a capability; agentic RAG adds control

Retrieval supplies evidence; orchestration decides when and how to retrieve

01 Basic RAG

- Question → retrieve top-k chunks
- Construct evidence-grounded prompt
- Generate answer with citations
- Best for predictable retrieval

02 Agentic RAG

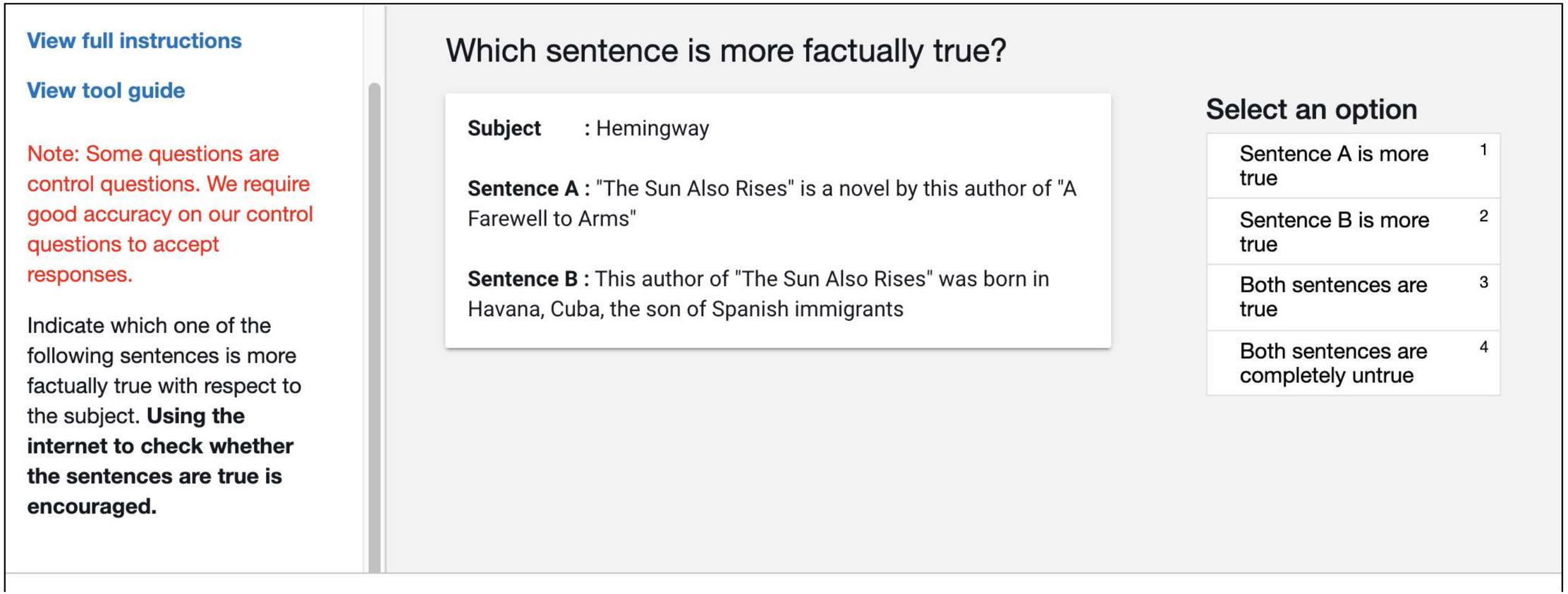
- Plan or decompose the query
- Retrieve and inspect evidence gaps
- Reformulate / rerank / compare sources
- Best when iterative search adds value

RAG answer: $p(y \mid x, D) \approx p_\theta(y \mid x, \text{top-k}(R(x, D)))$

R(x,D) retrieves evidence from corpus D; generation is conditioned on query x and retrieved passages.
Evidence annotation for grounded generation — Lewis et al., NeurIPS 2020 (arXiv:2005.11401)

Factual grounding: compare claims with evidence

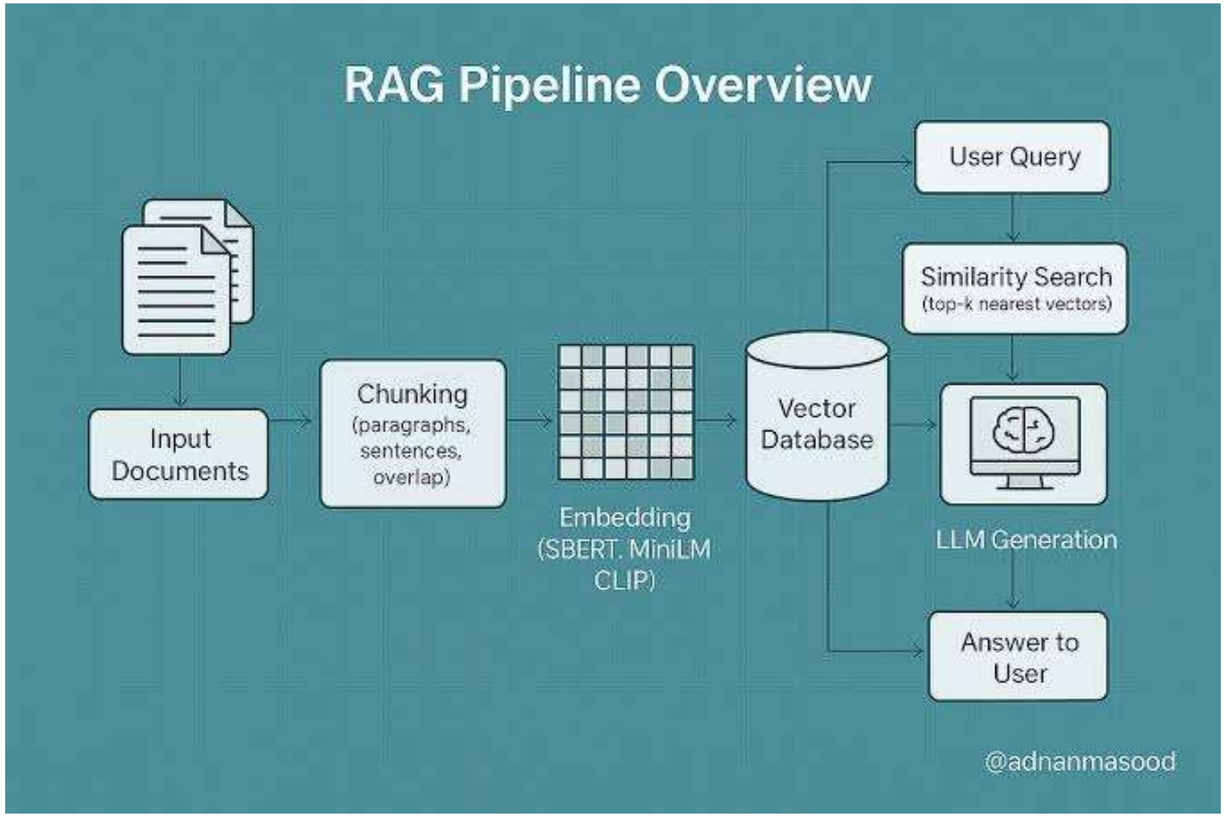
Compare factual claims against external evidence before choosing an answer



Evidence annotation for grounded generation — Lewis et al., NeurIPS 2020 (arXiv:2005.11401)

RAG pipeline: from documents to grounded answers

A retrieval system has an offline indexing path and an online query path



01 Offline indexing

- Parse PDFs, web pages and records
- Chunk with overlap and metadata
- Embed chunks and store vectors
- Maintain source IDs and versioning

02 Online retrieval

- Embed user query
- Vector / hybrid search + filters
- Rerank and assemble context
- Generate with citations and abstention

Code 2: a compact RAG function

The retriever and LLM are configured separately; preserve provenance with every chunk

PYTHON / IMPLEMENTATION PATTERN

```
def answer_with_rag(question: str, retriever, llm) -> dict:
    docs = retriever.invoke(question)
    context = "\n\n".join(
        f"[{i}] {doc.page_content}"
        for i, doc in enumerate(docs, start=1)
    )
    prompt = f"""Answer only from the evidence.
If evidence is insufficient, say so.
Cite evidence IDs such as [1], [2].

QUESTION: {question}
EVIDENCE:
{context}
"""
    answer = llm.invoke(prompt)
    return {
        "answer": answer.content,
        "sources": [doc.metadata for doc in docs]
    }
```

Production additions

1. Metadata filters for tenant / access scope
2. Hybrid lexical + vector retrieval
3. Reranking and context-window budgeting
4. Citation validation against source spans
5. Prompt-injection screening of retrieved content
6. Evaluate recall@k and groundedness

Why LangGraph?

Real tasks require branching, cycles, persistence and human pauses—not only linear chains

01 Stateful execution

- Nodes read and update shared state
- State carries goal, evidence and status

02 Conditional edges

- Route based on tool calls or review
- Branch on validation and policy

03 Cycles + recovery

- Retry, re-plan and revise
- Checkpoint and resume after interruption

04 Human interrupts

- Pause before sensitive actions
- Resume only after explicit decision

START › NODE › ROUTER › TOOL / REVIEW › END

LangGraph mental model: state, nodes and edges

Think of a graph as a program operating over evolving application state

01 State

- Typed data carried between steps
- Example: goal, messages, evidence, review flag

02 Nodes

- Functions that perform work
- Examples: planner, retriever, tool executor

03 Edges

- Rules selecting the next node
- Can be fixed or conditional

$$G = (V, E) \qquad S_{t+1} = F_v(S_t)$$

V is the set of nodes; E defines legal transitions. A node transforms state; an edge chooses the next computation.

Code 3: define a typed graph state

Make the state explicit, minimal, serializable and traceable

PYTHON / IMPLEMENTATION PATTERN

```
from typing import TypedDict
from langchain_core.messages import AnyMessage

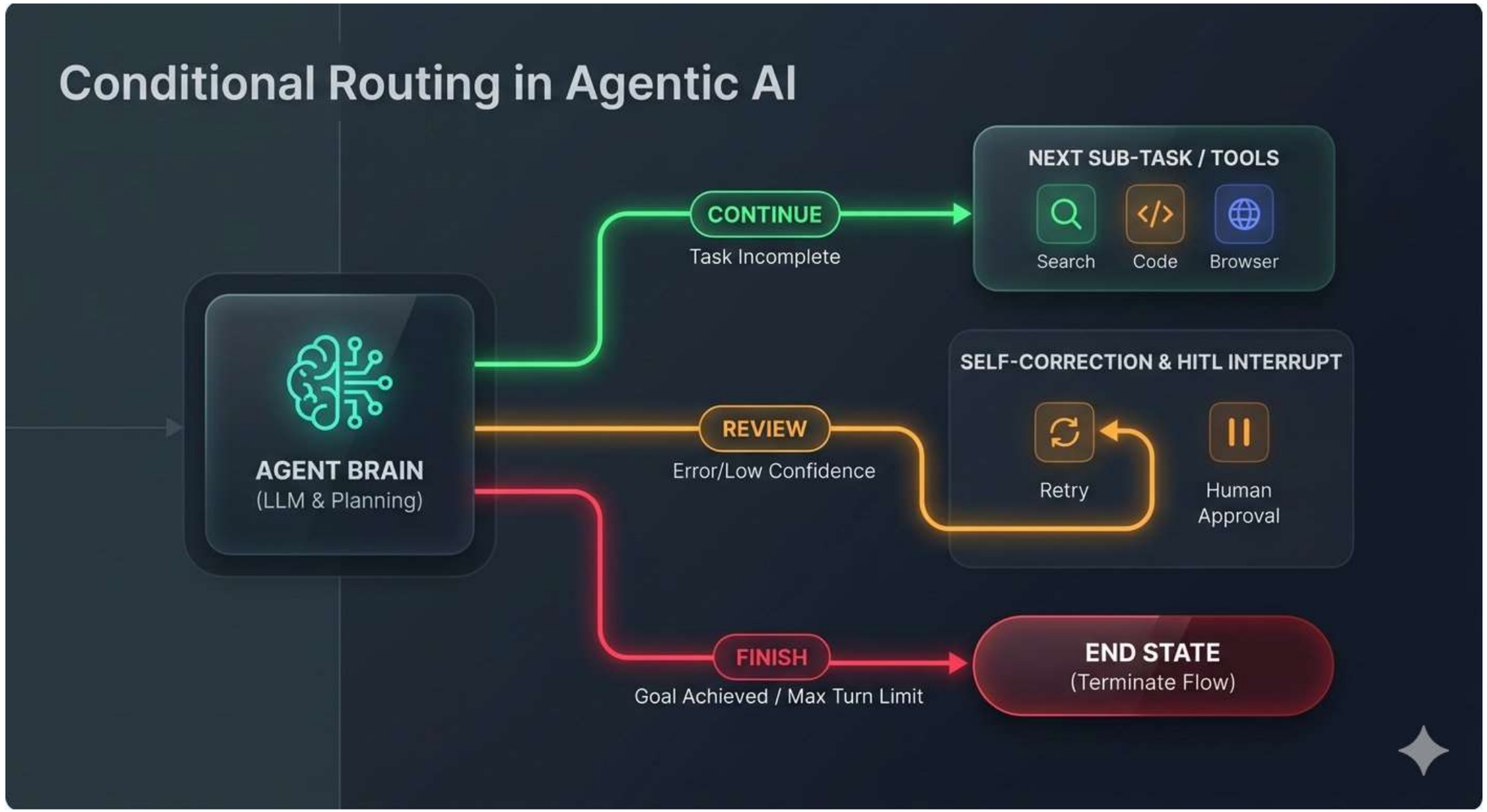
class AgentState(TypedDict):
    goal: str
    messages: list[AnyMessage]
    evidence: list[dict]
    draft: str
    needs_review: bool
    status: str
    step_count: int
```

State design rules

1. Typed: define explicit field types.
2. Small: store large files by reference.
3. Serializable: checkpoint backend can encode it.
4. Traceable: include IDs and provenance.
5. Versionable: plan schema migration.
6. Sensitive: never persist credentials in state.

Conditional routing: continue, review or finish

A router converts state into a controlled next-step decision



Conditional routing: components and controls

A router converts state into a controlled next-step decision

01 Agent node

- Model proposes response or tool call
- Node writes result into state

02 Router

- Inspect tool_calls, status and risk
- Return a finite route label

03 Tool / review node

- Execute only authorized tools
- Pause for human approval when required

04 End

- Verify output and evidence
- Finish or safe-stop

route(S) ∈ {tools, review, finish, retry, stop}

The router is deterministic application logic where possible; it must not bypass authorization.

Code 4: tools, model and graph state

Core pattern: model → tools → model → END; pin compatible package versions

PYTHON / IMPLEMENTATION PATTERN

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain_core.tools import tool

@tool
def lookup_lab(name: str) -> str:
    """Read status for an approved lab."""
    return lab_status_api(name)

tools = [lookup_lab]
llm = chat_model.bind_tools(tools)

class State(TypedDict):
    messages: Annotated[list, add_messages]

def agent(s):
    return {"messages": [llm.invoke(s["messages"])]}
```

Before running

1. Install langgraph + langchain-core.
2. Configure a real chat model and credentials.
3. Add authorization before ToolNode execution.
4. Test read-only tools first.
5. Add recursion / step limits and tracing.

Code 4: routing and graph wiring

Core pattern: model → tools → model → END; pin compatible package versions

PYTHON / IMPLEMENTATION PATTERN

```
def route(s):
    return "tools" if s["messages"][-1].tool_calls else "end"

g = StateGraph(State)
g.add_node("agent", agent)
g.add_node("tools", ToolNode(tools))
g.add_edge(START, "agent")
g.add_conditional_edges("agent", route,
                        {"tools": "tools", "end": END})
g.add_edge("tools", "agent")
app = g.compile()
```

Before running

1. Install langgraph + langchain-core.
2. Configure a real chat model and credentials.
3. Add authorization before ToolNode execution.
4. Test read-only tools first.
5. Add recursion / step limits and tracing.

Persistence: checkpoint after meaningful steps

Checkpointing enables recovery, long-running tasks and stateful conversations

01 Checkpoint

- Persist graph state at safe boundaries
- Associate with thread / task identity

02 Resume

- Reload latest checkpoint
- Continue from the saved execution point

03 Operational design

- Use durable storage in production
- Version state schemas
- Keep secrets in a secret manager

Retrieve › Draft › Checkpoint › Pause / failure › Resume

Human-in-the-loop: interrupt before high-impact actions

Approval is a control boundary—not a cosmetic confirmation dialog

01 Prepare

- Show proposed action and evidence
- Explain scope, impact and reversibility

02 Pause

- Persist graph state
- Wait for an authorized reviewer

03 Decide

- Approve, reject or request changes
- Record identity and rationale

04 Resume

- Continue only on approval
- Abort or compensate on rejection

Execute(a) = 1 only if Policy(a, user) ∧ Approval(a)

Approval should bind to the exact action payload, identity, timestamp and resource.

Multi-agent systems: when decomposition helps

Use multiple agents only when specialization or parallelism justifies coordination cost

01 Supervisor

- Coordinator decomposes and delegates
- Owns final integration and verification

02 Sequential

- Researcher → analyst → reviewer
- Each stage consumes a typed artifact

03 Parallel

- Independent subtasks run concurrently
- Merge only after validating outputs

04 Shared graph

- Specialists update controlled state
- Define ownership and conflict resolution

$$T_{parallel} \approx \max_i T_i + T_{coord} \qquad \text{Speedup} \leq N$$

Coordination, communication and merge time reduce ideal parallel speedup.

MCP vs A2A: tools are not other agents

Choose the protocol boundary by responsibility: capability access versus delegated work

01 MCP

- Agent ↔ tool / resource server
- Search papers, read files, query telemetry
- Standardizes capability discovery and invocation

02 A2A

- Agent ↔ agent service
- Delegate a specialist task to another agent
- Coordinates task hand-off and agent collaboration

MCP does not turn a tool into an autonomous peer; A2A is for delegated agent-level tasks.
MCP: Agent → Capability | A2A: Agent → Agent

Code 5: orchestrate specialist roles as graph nodes

A supervisor pattern routes work to specialist functions and a final reviewer

PYTHON / IMPLEMENTATION PATTERN

```
def planner(state):
    return {"plan": make_plan(state["goal"])}

def researcher(state):
    return {"evidence": search_sources(state["plan"])}

def reviewer(state):
    return {"review": check_evidence(state["evidence"])}

def route(state):
    if state["review"].needs_more_evidence:
        return "researcher"
    return "finish"

# START → planner → researcher → reviewer
# reviewer --more evidence--> researcher
# reviewer --verified-------> END
```

Design questions

1. What state is shared?
2. Which artifacts cross agent boundaries?
3. Can independent work run in parallel?
4. Who owns final verification?
5. What prevents delegation loops?
6. How are tool budgets enforced?

Inference stack: the runtime beneath the agent

The application, scheduler, model engine and accelerator jointly determine performance

01 Agent application

- Prompts, tools, policies and state
- May issue several model calls per task

02 Gateway + scheduler

- Authentication, routing and rate limits
- Queueing, batching and admission

03 Model engine + hardware

- Prefill, decode and attention kernels
- GPU compute, HBM and network

04 Task-level metrics

- End-to-end latency and success rate
- Throughput, memory and cost per task

Agent app › API gateway › Scheduler › Model engine › GPU / memory

Inference: prefill and decode

Autoregressive serving has two phases with different bottlenecks

01 Prefill

- Processes the input prompt in parallel
- Builds initial key/value tensors
- Often compute-intensive for long contexts

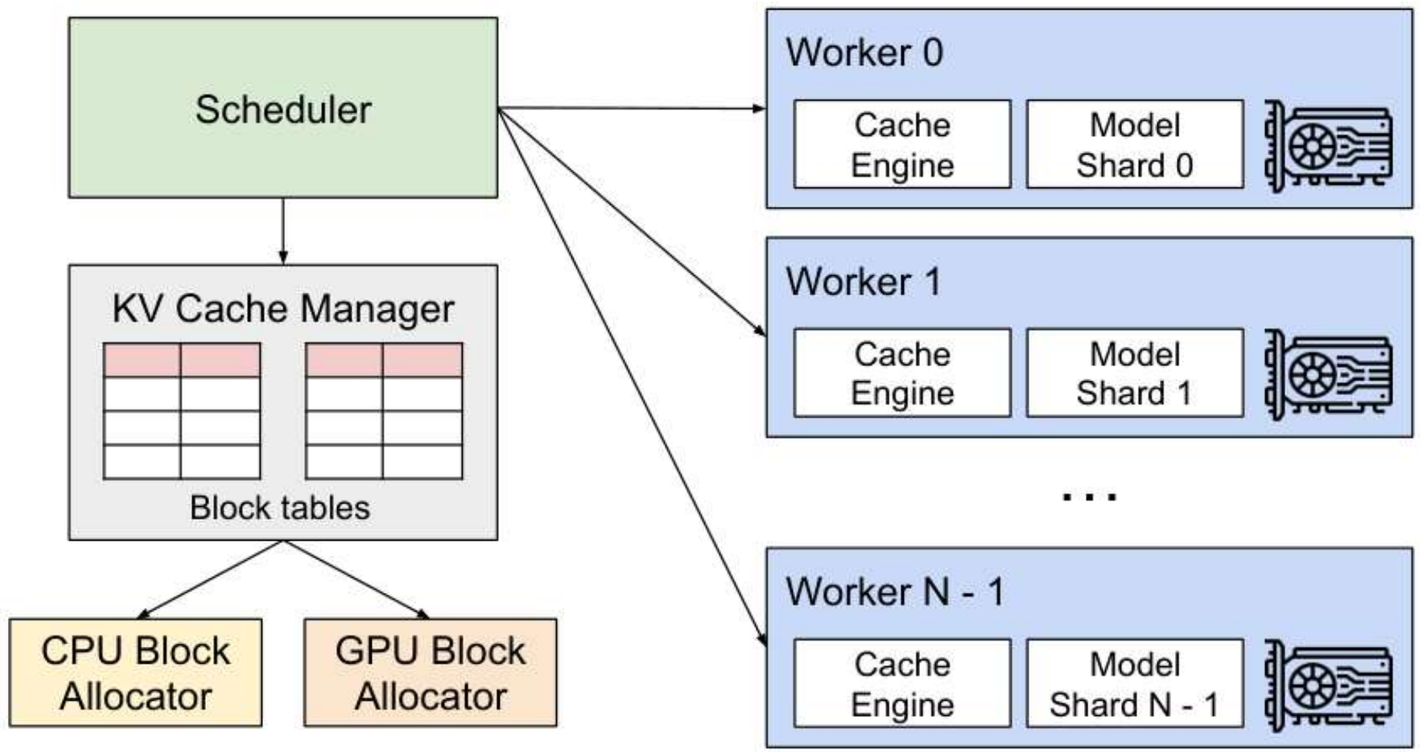
02 Decode

- Generates one token per sequence step
- Reuses cached keys and values
- Often memory-bandwidth sensitive

$T_{total} \approx T_{queue} + T_{prefill} + N_{out} \cdot T_{decode/token}$
For an agent, add tool latency and repeated model turns: $T_{task} = \Sigma T_{model} + \Sigma T_{tool} + T_{queue}$.

KV cache: memory grows with context and concurrency

The cache stores attention keys and values so previous tokens need not be recomputed



vLLM system overview: PagedAttention KV-cache management — Kwon et al., SOSP 2023 (arXiv:2309.06180)

Why cache?

- At each decode step, attention needs prior K and V
- Reusing them avoids recomputing the full prefix
- Cache grows with sequence length and active requests

Paged Attention

- Allocate KV memory in fixed-size blocks
- Map logical token blocks to physical blocks
- Reduce fragmentation and enable sharing where safe

$$M_{KV} \approx 2 \cdot L \cdot B \cdot T \cdot H_{kv} \cdot d_h \cdot \text{bytes/element}$$

L layers, B active sequences, T cached tokens, H_{kv} KV heads, d_h head dimension; exact layout depends on engine.

Continuous batching and serving metrics

Throughput gains must be balanced against tail latency and fairness

01 Continuous batching

- Admit new sequences as others finish
- Avoid waiting for a fixed batch to complete
- Scheduler manages token budgets

02 Measure the right metrics

- TTFT: time to first token
- TPOT: time per output token
- P95/P99 latency under load
- Task latency includes tools, retries and approvals

Throughput = completed requests / second **TPOT ≈ (t_last − t_first)/(N_out − 1)**

Report both throughput and tail latency; average latency can hide poor user experience.

Local inference with Ollama

Local model serving is useful for prototyping, privacy-sensitive experiments and reproducibility

SHELL + PYTHON

```
# Pull a model
ollama pull qwen2.5:7b

# Run a local prompt
ollama run qwen2.5:7b \
  "Explain tool calling in one paragraph."

# Local API (default)
# http://localhost:11434

# Python client sketch
from ollama import chat
response = chat(
    model="qwen2.5:7b",
    messages=[{"role": "user", "content": "Hello"}]
)
print(response.message.content)
```

System considerations

1. Match model size to RAM / VRAM.
2. Quantization trades precision for memory/speed.
3. Tool-calling reliability varies by model.
4. Record model, quantization and runtime versions.
5. Evaluate before production routing.

Model routing: match model to task

Select an inference route using complexity, latency, cost and risk

01 Fast route

- Classification and extraction
- Short summaries and routine tool selection
- Lower latency / cost

02 Strong route

- Complex planning and synthesis
- Ambiguous evidence or multi-step reasoning
- Higher capability; higher cost

03 Verification

- Validate output schema and policy
- Escalate uncertain / high-impact tasks
- Fallback must preserve authorization

$$m^* = \arg\min_{m} \; [\, \alpha \cdot \text{Latency}(m) + \beta \cdot \text{Cost}(m) + \gamma \cdot \text{Risk}(m) \,]$$

Choose weights from service requirements; impose hard constraints on quality, privacy and safety.

MCP: standardizing tool and context access

MCP separates the AI host from external capability implementations

01 Host

- AI application and user interaction
- Controls connected servers and exposed capabilities

02 Client

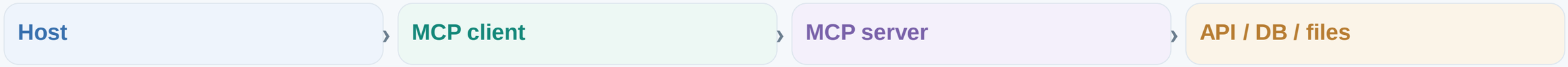
- Protocol client inside the host
- Negotiates version and capabilities; sends requests

03 Server

- Exposes tools, resources and prompts
- Validates arguments and enforces server-side access

04 Backend

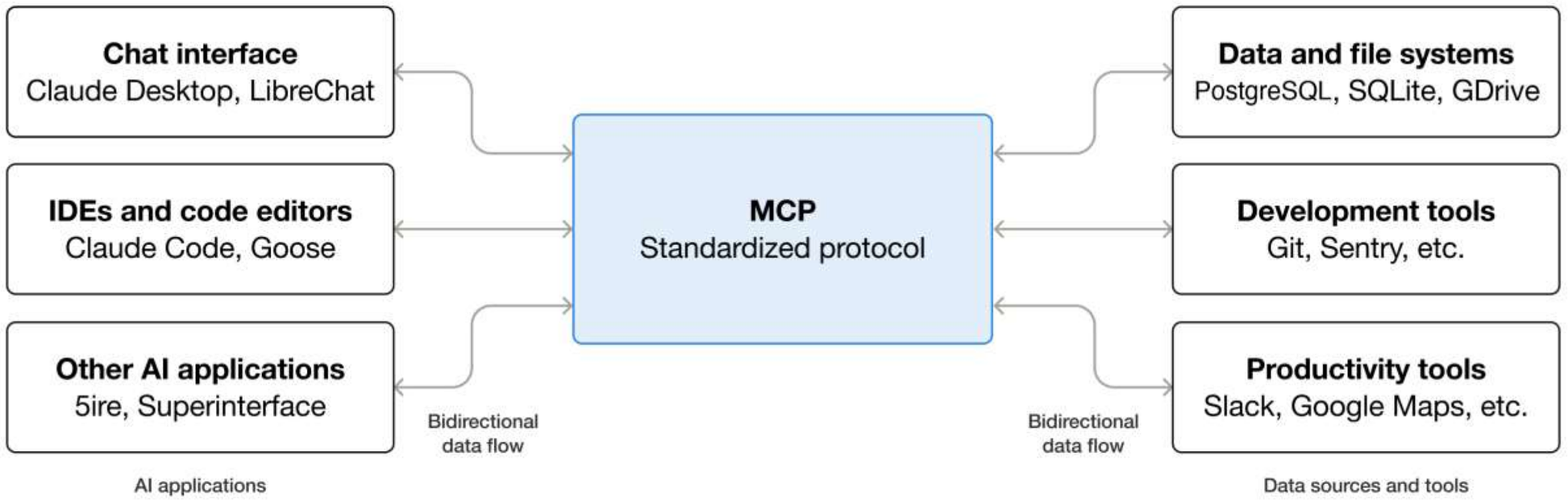
- APIs, databases, files or services
- Performs the actual operation



MCP architecture: host, clients and servers — modelcontextprotocol.io

MCP ecosystem: applications, data and tools

AI applications connect to data sources and tools through MCP



MCP architecture: host, clients and servers — modelcontextprotocol.io

MCP primitives: tools, resources and prompts

Three distinct server primitives expose actions, information and reusable interaction patterns

01 Tools

- Executable actions
- Examples: search, calculate, update record
- Invoked through a structured call

02 Resources

- Readable context
- Examples: documents, records, telemetry
- Identified and retrieved by URI

03 Prompts

- Reusable templates
- Examples: review checklist, report structure
- Help standardize user/model workflows

MCP standardizes the interface; the host and server still own policy, consent and authorization.
Tools do • Resources inform • Prompts guide

Code 6: build a small MCP tool server

FastMCP can expose a typed Python function as a discoverable tool

PYTHON / IMPLEMENTATION PATTERN

```python
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("Lab Status Server")
LABS = {
    "robotics": "operational",
    "fpga": "maintenance"
}

@mcp.tool()
def get_lab_status(lab: str) -> str:
    """Return status for a known laboratory."""
    key = lab.strip().lower()
    if key not in LABS:
        return "Unknown lab"
    return LABS[key]

if __name__ == "__main__":
    mcp.run(transport="stdio")
```

To run the example

1. Install a compatible MCP Python SDK.
2. Save as server.py.
3. Configure an MCP host/client to launch it.
4. Host discovers get_lab_status and its schema.
5. Production: identity, permissions, audit and real backend.
6. Pin SDK and protocol version.

MCP integration: discovery is not authorization

A tool being discoverable does not mean every user may execute it

01 Initialize + discover

- Negotiate protocol and capabilities
- List available tools and schemas

02 Validate + authorize

- Validate arguments against schema
- Check identity, role, scope and resource

03 Execute + audit

- Apply timeout and rate limit
- Log actor, action, result and timestamp

04 Return safely

- Return structured output
- Treat tool output as untrusted external data

Allow(call) = SchemaValid ∧ Authenticated ∧ Authorized ∧ InScope

MCP is an interoperability protocol—not a security boundary by itself.

Case study: self-healing DevOps workflow

LangGraph + MCP can connect telemetry, repositories, tests and ticketing systems

01 Observe + diagnose

- Read alert and telemetry
- Inspect logs and recent deployment changes
- Form a remediation hypothesis

02 Propose + verify

- Draft a patch in a branch
- Run tests in an isolated sandbox
- Collect test and security evidence

03 Review + release

- Human reviews diff and evidence
- Approve merge/deploy explicitly
- Monitor post-deployment health

Alert › Diagnose › Propose patch › Sandbox tests › Human approval › Deploy

Sandbox execution: prove the patch before approval

Autonomous code modification needs an isolated execution boundary

01 Isolation

- Ephemeral container or VM
- No production credentials by default
- Network and filesystem restrictions

02 Verification

- Target test + regression suite
- Static analysis and dependency scan
- Resource/time limits

03 Approval

- Review exact diff and logs
- Approve merge/deploy explicitly
- Rollback and kill switch available

Promote(patch) = TestsPass ∧ SecurityPass ∧ HumanApproval

A successful sandbox run is evidence—not a guarantee of production safety.

From notebook to service: the MLOps layer

Production agents need APIs, durable state, scaling, secrets and observability

01 API layer

- Invoke, batch and streaming interfaces
- Authentication and request validation

02 State + runtime

- Durable checkpoint backend
- Containerized graph runtime and autoscaling

03 Operations

- Secret manager and policy service
- Metrics, logs, traces and alerting

Client › API gateway › Agent service › Checkpoint DB › Observability

Code 7: expose an agent as an API

A service wrapper turns a compiled graph into an endpoint; secure it at the boundary

PYTHON / IMPLEMENTATION PATTERN

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(title="Research Agent API")

class TaskRequest(BaseModel):
    goal: str
    thread_id: str

@app.post("/agent/invoke")
async def invoke_agent(req: TaskRequest):
    config = {
        "configurable": {"thread_id": req.thread_id}
    }
    result = await agent_graph.ainvoke(
        {"goal": req.goal},
        config=config
    )
    return {
        "status": result["status"],
        "answer": result["draft"]
    }
```

Deployment checklist

1. Authenticate and authorize every request.
2. Apply quotas, timeouts and payload limits.
3. Use durable checkpointer for thread state.
4. Validate outputs against a response schema.
5. Redact secrets from logs and traces.
6. Pin and test framework versions.

Observability: trace the complete trajectory

Trace the agent as a workflow—not merely as a sequence of LLM calls

01 Trace identity

- Run ID, user and thread ID
- Model, prompt and tool versions

02 Execution path

- Node transitions and route decisions
- Tool name, validated arguments and result IDs

03 Performance

- Queue, model and tool latency
- Tokens, retries and cost per task

04 Governance

- Policy decisions and approvals
- Outcome, exceptions and audit record

Planner › LLM call › MCP tool › Synthesis › Evaluation

Evaluation: measure task quality and tool behaviour

Use repeatable datasets and trace-level checks—not anecdotal demonstrations

01 Task metrics

- Success rate = successful tasks / total tasks
- Exact match, F1 or task-specific utility
- Evidence support / citation precision

02 Tool metrics

- Tool selection accuracy
- Argument validity and execution success
- Policy violation and retry rates

03 Systems metrics

- P50 / P95 / P99 task latency
- Tokens and cost per successful task
- Recovery rate under injected failures

$$\text{Score} = w_1 \cdot \text{Success} + w_2 \cdot \text{Groundedness} - w_3 \cdot \text{Cost} - w_4 \cdot \text{Violations}$$

Weights are task-specific; safety constraints should be hard gates, not merely soft penalties.

Governance and incident response

Controls should exist before, during and after execution

01 Before execution

- Identity, scope and least privilege
- Tool allowlist and schema validation
- Threat modelling and red-team tests

02 During execution

- Step, token, time and cost budgets
- Sandbox, policy checks and approval gates
- Prompt-injection-aware data handling

03 After execution

- Output validation and audit trail
- Rollback, incident response and recovery
- Review failures and update evaluations

Risk exposure ∝ Autonomy × Privilege × Impact

Reduce exposure by constraining authority, limiting action scope and requiring approval for irreversible operations.

Putting it together: a production agent architecture

The design unit is the complete task trajectory—from user goal to verified outcome

01 Interaction + control

- User goal, constraints and identity
- LangGraph state + conditional routing
- Bounded ReAct loop and checkpointing

02 Intelligence + capability

- Hosted or local LLM inference
- RAG retrieval with provenance
- MCP tools; A2A delegation where justified

03 Trust + operations

- Authorization and human approvals
- Sandbox and output validation
- Tracing, evaluation, SLOs and incident response

User / goal › Graph controller › LLM + RAG › MCP tools › Verify + approve › Outcome

References and implementation reading

Research papers, protocol specifications and official engineering documentation

01 Yao, S. et al. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.

02 Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020. arXiv:2005.11401.

03 LangChain. LangGraph documentation: StateGraph, persistence/checkpointing and human-in-the-loop interrupts. docs.langchain.com/oss/python/langgraph/

04 Model Context Protocol. Specification and architecture: tools, resources, prompts, authorization and transports. modelcontextprotocol.io/

05 Kwon, W. et al. (2023). Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. DOI: 10.1145/3600006.3613165.

06 vLLM documentation. PagedAttention, continuous batching, prefix caching and serving metrics. docs.vllm.ai/

07 NIST (2024). AI 600-1: Artificial Intelligence Risk Management Framework—Generative AI Profile. DOI: 10.6028/NIST.AI.600-1.

08 OWASP GenAI Security Project (2025). Top 10 for Agentic Applications; Agentic Security Initiative guidance. genai.owasp.org.

Implementation note: APIs evolve. Pin package and protocol versions, consult official documentation, and run regression tests before deployment.